



## PyQuest

# Information- & Aufgabenblatt

## Kapitel 12: Verschachtelte Listen

Listen in Listen – Tabellen, Raster und mehrdimensionale Arrays.

### Verschachtelte Listen – Grundlagen

Eine Liste kann **alles** enthalten – auch **andere Listen**. Eine „Liste von Listen“ nennt man **verschachtelte Liste** (in anderen Sprachen auch: **mehrdimensionales Array** oder **2D-Array**).

Damit lassen sich **Tabellen** oder **Raster** abbilden: Jede innere Liste ist eine **Zeile**. Ein gutes Beispiel ist eine Notentabelle: jede Zeile enthält die Noten einer Schülerin oder eines Schülers.

3 Schüler, je 3 Noten – `noten[0]` ist die komplette erste Zeile:

*Beispiel*

```
noten = [[2, 3, 1], [4, 2, 3], [1, 1, 2]]  
print(noten[0])
```

Um auf ein **einzelnes** Element zuzugreifen, brauchst du **zwei** Indizes hintereinander: **`liste[zeile][spalte]`**. Zuerst wählst du die Zeile (die äußere Liste), dann das Element in dieser Zeile (die innere Liste).

`noten[1][2]` ist die dritte Note der zweiten Schülerin:

*Beispiel*

```
noten = [[2, 3, 1], [4, 2, 3], [1, 1, 2]]  
print(noten[1][2])
```

**Aufgabe 1: Was liefert `noten[1][2]` bei `noten = [[2, 3, 1], [4, 2, 3], [1, 1, 2]]`?**

- ☐ a) 4
- ☐ b) 3
- ☐ c) 2



Mit `len()` bekommst du wie gewohnt die **Anzahl der Elemente**. Bei einer verschachtelten Liste liefert `len(liste)` die Anzahl der **Zeilen**, und `len(liste[0])` die Anzahl der Elemente in der **ersten Zeile** (also meist die Spaltenanzahl).

**Aufgabe 2:** In `sitzplan` steht eine verschachtelte Liste: 2 Reihen mit je 3 Namen. Gib aus:  
1. Die komplette erste Reihe (`sitzplan[0]`) 2. Den Namen in der zweiten Reihe, dritter Platz (`sitzplan[1][2]`) 3. Die Anzahl der Reihen (`len(sitzplan)`)

```
sitzplan = [ ["Ada", "Alan", "Grace"], ["Linus", "Tim", "Katie"] ]
```

Dein Code:

## Verschachtelte Listen verändern

Genau wie bei normalen Listen kannst du mit `liste[zeile][spalte] = neuer_wert` ein einzelnes Element **überschreiben**.

Das Element in Zeile 0, Spalte 1 wird überschrieben:

*Beispiel*

```
matrix = [[1, 2], [3, 4]]  
matrix[0][1] = 99  
print(matrix)
```

Um eine **komplette neue Zeile** hinzuzufügen, benutzt du `.append()` auf der **äußeren** Liste – du hängst dabei eine **ganze Liste** an.

Um dagegen ein Element **innerhalb** einer bestehenden Zeile zu ergänzen, rufst du `.append()` auf der **inneren** Liste auf: `matrix[1].append(...)`.

`append()` auf der äußeren Liste fügt eine neue Zeile hinzu:

*Beispiel*

```
matrix = [[1, 2], [3, 4]]  
matrix.append([5, 6])  
print(matrix)
```



**Aufgabe 3: Wie fügst du der Zeile `matrix[1]` ein zusätzliches Element hinzu, ohne eine neue Zeile zu erzeugen?**

- ☐ a) `matrix.append(wert)`
- ☐ b) `matrix[1].append(wert)`
- ☐ c) `matrix[wert].append(1)`

**Aufgabe 4: Gegeben ist `klassenzimmer` mit 2 Reihen. Ändere den Namen in Reihe 0, Platz 1 zu "Neu". Füge danach der Reihe 1 mit `.append()` den Namen "Zusatz" hinzu. Gib `klassenzimmer` am Ende aus.**

```
klassenzimmer = [["Ada", "Alan"], ["Grace", "Linus"]]
```

Dein Code:

## Mit Schleifen durchlaufen

Um **jedes Element** einer verschachtelten Liste zu besuchen, brauchst du **zwei ineinander verschachtelte** for-Schleifen – genau wie beim Einmaleins-Beispiel im Schleifen-Kapitel.

Die **äußere** Schleife läuft über die **Zeilen**, die **innere** Schleife läuft über die **Elemente** innerhalb der aktuellen Zeile.

Die äußere Schleife holt jede Zeile, die innere jedes Element darin:

*Beispiel*

```
matrix = [[1, 2, 3], [4, 5, 6]]
for zeile in matrix:
    for wert in zeile:
        print(wert, end=" ")
    print()
```



Das `print()` **ohne Argument** nach der inneren Schleife sorgt für den **Zeilenumbruch** – es beendet die aktuelle Zeile, bevor die nächste Zeile der Matrix beginnt.

**Aufgabe 5: Warum braucht man ZWEI verschachtelte for-Schleifen, um eine 2D-Liste komplett auszugeben?**

- ☐ a) Weil Python das so vorschreibt
- ☐ b) Die äußere holt jede Zeile, die innere jedes Element in der Zeile
- ☐ c) Eine Schleife reicht, zwei sind nur schneller

**Aufgabe 6: Gib die verschachtelte Liste `matrix` zeilenweise aus: In jeder Zeile stehen die Zahlen durch ein Leerzeichen getrennt, nach jeder Zeile folgt ein Zeilenumbruch.**

```
matrix = [[7, 8, 9], [1, 2, 3], [4, 5, 6]]
```

Dein Code:

## Verschachtelte Listen erstellen

Oft willst du eine verschachtelte Liste **programmatisch** erzeugen, statt sie von Hand auszuschreiben – zum Beispiel ein leeres Spielfeld. Dafür baust du mit einer **äußeren Schleife** Zeile für Zeile auf: In jedem Durchlauf erzeugst du eine **neue innere Liste** und hängst sie mit `.append()` an die äußere Liste an.

Zeile für Zeile eine neue innere Liste erzeugen und anhängen:

*Beispiel*

```
matrix = []
for zeile_nr in range(3):
    neue_zeile = [0, 0, 0]
    matrix.append(neue_zeile)
print(matrix)
```



**Achtung, klassische Falle!** Man könnte auf die Idee kommen, eine Zeile mit \* zu **vervielfachen**: `[[0, 0, 0]] * 3`. Das sieht auf den ersten Blick richtig aus – ist es aber **nicht**!

Dabei entstehen **drei Verweise auf dieselbe innere Liste**, keine drei unabhängigen Zeilen. Änderst du eine Zeile, ändern sich **alle**:

```
falsch = [[0, 0, 0]] * 3
falsch[0][0] = 9
print(falsch) → [[9, 0, 0], [9, 0, 0], [9, 0, 0]] 🤖
```

**Merke:** Für unabhängige Zeilen musst du sie **einzeln** in einer Schleife erzeugen (wie im Beispiel oben) – niemals mit \* vervielfachen.

### Aufgabe 7: Warum ist `[[0, 0]] * 3` gefährlich für eine Matrix?

- ☐ a) Es erzeugt zu wenige Zeilen
- ☐ b) Alle drei Zeilen zeigen auf dieselbe innere Liste – eine Änderung wirkt sich auf alle aus
- ☐ c) Python erlaubt das gar nicht

**Aufgabe 8: Erzeuge mit einer for-Schleife eine 3x4-Matrix (3 Zeilen, je 4 Elemente) namens `matrix`, komplett gefüllt mit dem Wert 0. Baue jede Zeile einzeln auf (keine \*-Vervielfachung!) und hänge sie mit `.append()` an. Gib `matrix` am Ende aus.**

*Dein Code:*

## Zeilen- und Spaltensummen

Eine typische Aufgabe bei Tabellen: die **Summe jeder Zeile** berechnen. Dafür läufst du mit der äußeren Schleife über die Zeilen und bildest für **jede Zeile** mit der inneren Schleife eine eigene Summe – die du danach in eine **Ergebnisliste** sammelst.

Für jede Zeile wird die Summe gebildet und gesammelt:

*Beispiel*

```
matrix = [[1, 2, 3], [4, 5, 6]]
zeilensummen = []
```



```
for zeile in matrix:
    summe = 0
    for wert in zeile:
        summe += wert
    zeilensummen.append(summe)
print(zeilensummen)
```

Eine **Spaltensumme** ist etwas trickreicher: Eine Spalte besteht aus dem Element an **derselben Position** in **jeder** Zeile. Für Spalte k brauchst du also aus **jeder** Zeile das Element `zeile[k]` – dafür reicht **eine** Schleife über die Zeilen.

Summe der Spalte mit Index 1 (zweite Spalte):

*Beispiel*

```
matrix = [[1, 2, 3], [4, 5, 6]]
spalten_index = 1
spaltensumme = 0
for zeile in matrix:
    spaltensumme += zeile[spalten_index]
print(spaltensumme)
```

**Aufgabe 9: Wie viele Schleifen brauchst du mindestens, um die Summe EINER bestimmten Spalte zu berechnen?**

- ☐ a) Keine
- ☐ b) Eine
- ☐ c) Zwei

**Aufgabe 10: Gegeben ist matrix (3 Zeilen à 3 Zahlen). Berechne mit verschachtelten Schleifen die Summe jeder Zeile und sammle die Ergebnisse in der Liste `zeilensummen`. Gib `zeilensummen` aus.**

```
matrix = [[2, 4, 6], [1, 3, 5], [7, 8, 9]]
```

*Dein Code:*



**Aufgabe 11: Berechne für dieselbe matrix die Summe der Spalte mit Index 1 (also der zweiten Spalte) und gib sie aus.**

```
matrix = [[2, 4, 6], [1, 3, 5], [7, 8, 9]]
```

Dein Code:

## Verschachtelte Listen anwenden

Jetzt kombinierst du alles: verschachtelte Schleifen, Positionen (i, j) und das **Merken des bisher besten Werts** (das kennst du schon von der Suche nach der größten Zahl in einer einfachen Liste).

Um bei einer Matrix nicht nur den **größten Wert**, sondern auch seine **Position** zu finden, läufst du mit `range(len(matrix))` über die Zeilen-**Indizes** und mit `range(len(matrix[i]))` über die Spalten-**Indizes** – so hast du beide Indizes zur Hand, wenn du den neuen Rekord findest.

Bei jedem neuen Rekord werden Wert UND Position gemerkt:

*Beispiel*

```
matrix = [[3, 7], [9, 1]]
groesster = matrix[0][0]
for i in range(len(matrix)):
    for j in range(len(matrix[i])):
        if matrix[i][j] > groesster:
            groesster = matrix[i][j]
print(groesster)
```



**Aufgabe 12:** Finde in `matrix` den größten Wert und seine Position. Starte mit `groesster_wert = matrix[0][0]`, `zeile_index = 0`, `spalte_index = 0`. Durchlaufe mit Indizes (`range(len(matrix))`) und `range(len(matrix[i]))`) und aktualisiere bei jedem neuen Rekord alle drei Variablen.

Gib am Ende `groesster_wert` aus, dann `zeile_index` und `spalte_index` (mit `print(zeile_index, spalte_index)` in einer Zeile).

```
matrix = [[3, 7, 2], [9, 1, 8], [4, 6, 5]]
```

Dein Code:

Eine Notentabelle wie ganz am Anfang des Kapitels lässt sich jetzt auswerten: Für **jede Zeile** (jeden Schüler) berechnest du den **Durchschnitt** – Zeilensumme geteilt durch die Anzahl der Noten in dieser Zeile.

**Aufgabe 13:** In `noten` steht für 3 Schüler je eine Zeile mit Noten. Berechne für jede Zeile den **Durchschnitt** (Zeilensumme / Anzahl Noten in der Zeile) und gib ihn gerundet auf 2 Nachkommastellen aus (`round(durchschnitt, 2)`) – einen Durchschnitt pro Zeile.

```
noten = [[2, 3, 1], [4, 2, 3], [1, 1, 2]]
```

Dein Code: